TradeMarkia AI Intern Assessment - 20 Newsgroups

Semantic Search with Fuzzy Clustering and Semantic Cache

Project Report - Documentation

Submitted by

Bhumit Goyal (23BCI0137)

Email: bhumit.goyal2023@vitstudent.ac.in

Email (Personal) : bhumitgoyal.bg@gmail.com

B.Tech Computer Science Engineering

VIT Vellore

Internship Project — Trademarkia AI

1. Abstract

I developed a **full-stack semantic search system** built on top of the **20 Newsgroups dataset**, where my goal was to design and evaluate an intelligent document retrieval pipeline. In this project, I combined **dense vector embeddings, a persistent vector database, fuzzy clustering, and a semantically aware caching layer** so that the system could retrieve documents even when queries were paraphrased or phrased differently.

To make the system usable end-to-end, I built a **FastAPI backend** to handle the query processing and API endpoints, and a **Streamlit frontend** so users could easily interact with the search interface. I also containerized the entire system using **Docker**, and created a **single-command launcher** so the whole pipeline — including the backend and frontend — could start with minimal setup.

2. Introduction

I started by identifying a limitation of **traditional keyword-based search** — it cannot capture semantic similarity. For example, queries like *"firearms legislation"* and *"gun control debate"* talk about the same topic but may not match because they don't share exact keywords. To solve this, I used **neural embedding models** that convert text into vectors where semantically similar texts are closer together.

However, I also noticed two practical issues with pure vector search: **high latency** from embedding and searching large vector databases, and **redundant computation** when users ask paraphrased versions of the same query.

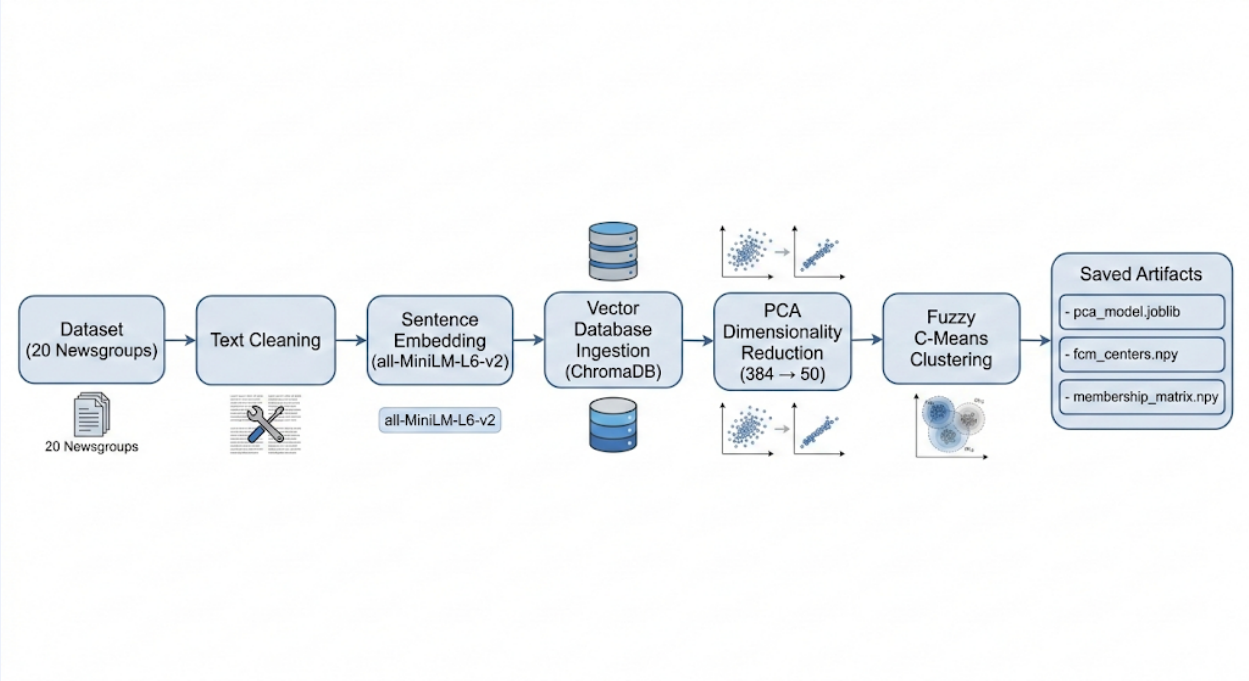
To address this, I implemented **Fuzzy C-Means clustering** to capture multi-topic relationships in the corpus, and built a **semantic cache** that uses cluster membership to quickly serve results for similar queries without running full retrieval again.

3. System Architecture

The system is split into an offline pipeline and an online query path.

3.1 Offline Pipeline

The offline pipeline is responsible for building all search artefacts from the raw 20 Newsgroups dataset. It runs once before the system is deployed.

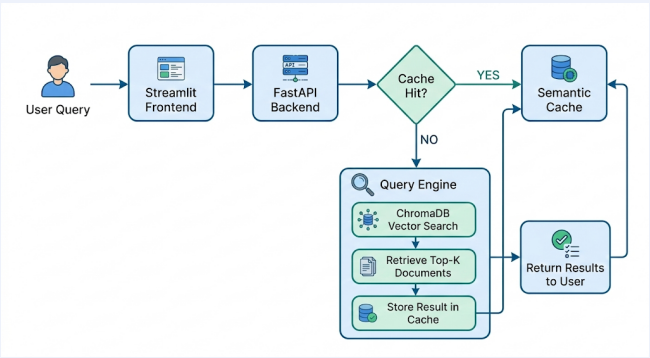


Pipeline stages:

1. Dataset download from UCI ML Repository.
2. Text cleaning: removal of newsgroup headers, quoted lines, signatures, and noise.
3. Embedding: sentence-transformers (all-MiniLM-L6-v2) producing 384-d normalised vectors.
4. Vector DB ingestion: cleaned texts + metadata stored in ChromaDB.
5. PCA dimensionality reduction (384 → 50 dims) followed by L2 normalisation.
6. Fuzzy C-Means clustering over reduced embeddings; artefacts saved to disk.

3.2 Online Query Path

At query time, the following flow executes:



7. User submits query through the Streamlit UI.
8. FastAPI receives the request and invokes the semantic cache.
9. Cache embeds the query, predicts FCM cluster memberships, and scans cluster-matched cache entries.
10. Cache hit: returns stored result with matched query and similarity score.
11. Cache miss: query engine embeds the query, calls ChromaDB for top-k results, returns documents, and stores the result in cache.

3.3 Component Overview

Component	File	Responsibility
Corpus Prep	src/corpus_prep.py	Text cleaning, embedding, ChromaDB ingestion
Fuzzy Clustering	src/fuzzy_clustering.py	PCA, FCM, k-selection analysis, artefact saving
Semantic Cache	src/semantic_cache.py	LRU cache with cluster-gated cosine lookup
Query Engine	src/query_engine.py	Query embedding + ChromaDB retrieval
API Server	src/main.py	FastAPI endpoints, lifecycle management
Frontend	frontend/app.py	Streamlit UI
Pipeline Script	scripts/run_pipeline.py	Orchestrates offline pipeline stages
Launcher	start.py	One-command system startup

4. Dataset: 20 Newsgroups

The categories cover a wide range of discussions, including **science** (like *sci.space* and *sci.med*), **politics** (such as *talk.politics.guns*), **religion** (*talk.religion.misc*, *alt.atheism*), **sports** (*rec.sport.hockey*, *rec.sport.baseball*), and **technology** (*comp.graphics*, *comp.os.ms-windows.misc*). This diversity made it a good dataset for testing how well my semantic search system could handle different topics and retrieve relevant documents.

Property	Value

Total documents	~18,846
Number of categories	20
Typical document length	50–500 words (after cleaning)
Source	UCI ML Repository
Primary challenge	Noisy headers, signatures, quoted replies

The dataset presents a realistic retrieval challenge: posts are noisy, multi-topical, and often contain irrelevant metadata. This makes it a good benchmark for evaluating both embedding quality and clustering robustness.

5. Text Preprocessing

5.1 Cleaning Heuristics

Newsgroup posts contain significant noise that degrades embedding quality. A rule-based cleaning pipeline is applied before embedding:

- Remove Usenet/email headers (From:, Subject:, Lines:, etc.).
- Strip quoted lines beginning with > (reply chains).
- Remove lines that are too short (< 3 tokens) to carry semantic signal.
- Remove signature blocks (--) and metadata residues.
- Strip URLs and email addresses.

The result is a clean body of prose that produces more meaningful sentence embeddings.

5.2 Why Cleaning Matters

Without cleaning, the sentence-transformer would embed noise tokens alongside content. Quoted reply text in particular introduces false similarity between documents that share a quoted thread rather than genuine topical relevance. Removing this noise isolates the document's original contribution.

6. Embeddings

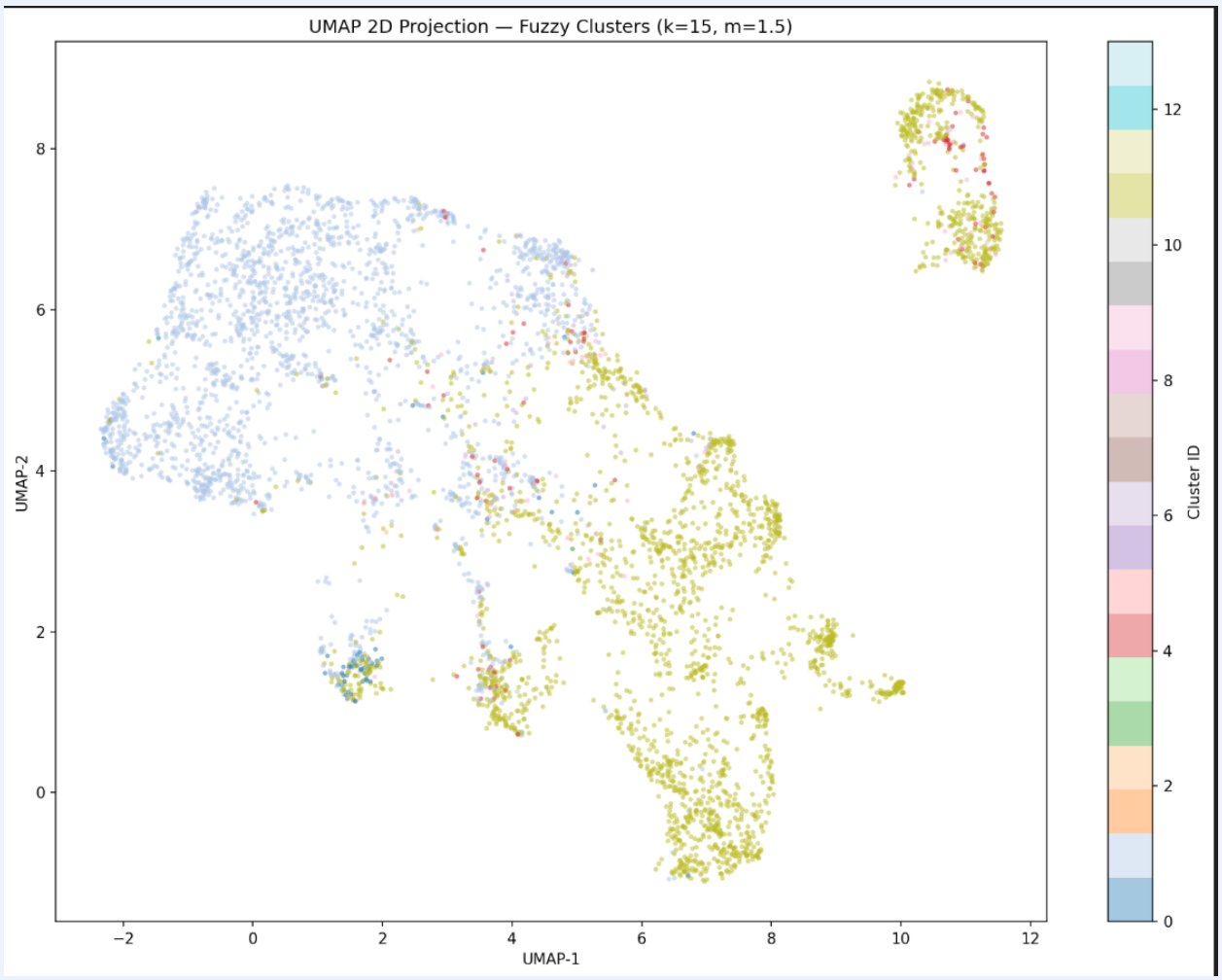
6.1 Model: all-MiniLM-L6-v2

I embedded all documents and queries using the **all-MiniLM-L6-v2 sentence-transformer model**. I chose this model because it produces **384-dimensional dense embeddings** that capture semantic meaning effectively.

Another advantage is that the vectors are **L2-normalised**, which allows me to compute **cosine similarity using a simple dot product**, making retrieval faster. The model also performs well on **semantic similarity and retrieval benchmarks**, while maintaining a **small memory footprint (~80MB)** and **fast CPU inference**, which made it suitable for my system.

6.2 Batch Processing

Documents are embedded in batches with progress tracking via tqdm. Embeddings are stored as a NumPy array (.npy) on disk to avoid recomputation across runs. Corpus metadata (newsgroup label, doc_id, clean_text) is persisted as a Parquet file for efficient columnar access.



7. Vector Database: ChromaDB

7.1 Design Choice

ChromaDB was selected as the vector store for the following reasons:

- Persistent local storage without requiring a separate database server.
- Built-in cosine similarity search via its internal HNSW index.
- Metadata filtering support (newsgroup category, doc_id).
- Simple Python API that integrates cleanly with the pipeline.

7.2 Collection Schema

Each document is stored with:

- Embedding vector (384-d float).
- Document text (clean_text).
- Metadata: newsgroup category, document ID.

At query time, a top-k query is issued: the k nearest vectors by cosine distance are returned along with their metadata and distance scores.

---

8. Fuzzy Clustering

8.1 Motivation for Soft Assignments

Standard k-means clustering assigns each document to exactly one cluster. This is inadequate for newsgroup posts, which frequently discuss multiple topics simultaneously (e.g., a post about "guns and self-defence laws" spans both firearms and politics). Fuzzy C-Means (FCM) addresses this by assigning each document a vector of membership probabilities across all clusters.

Example membership vector for a single document:

```
cluster  3  →  0.72
cluster 11  →  0.18
cluster  7  →  0.10
```

This richer representation is used to power the cluster-gated semantic cache lookup.

8.2 Dimensionality Reduction (PCA)

Before running clustering, I first reduced the **384-dimensional embeddings** to **50 principal components** using **scikit-learn PCA**, and then applied **L2 normalisation**.

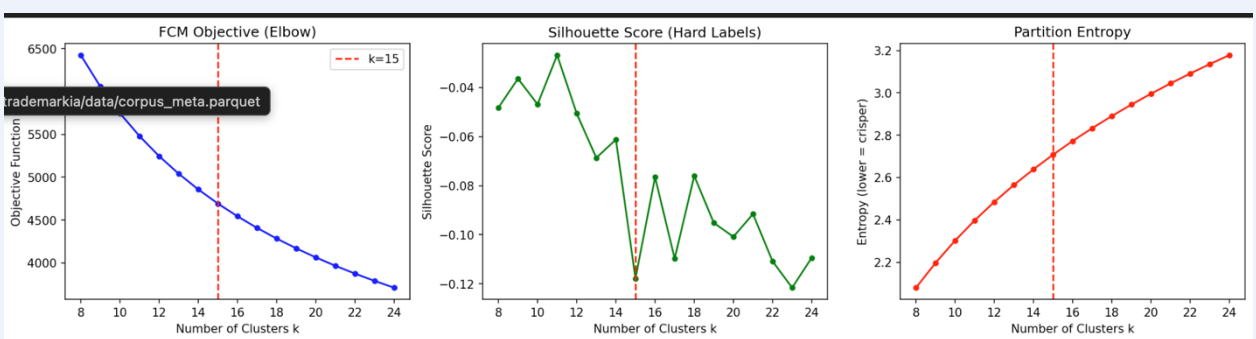
I did this for two main reasons. First, it **removes noisy dimensions with low variance**, which helps make the **Fuzzy C-Means clustering more stable during convergence**. Second, it **reduces the computational cost** of clustering from roughly $O(n \times 384)$ to $O(n \times 50)$, making the algorithm faster.

I also saved the trained PCA model as `pca_model.joblib`, so that I can **apply the same transformation to query embeddings at runtime** without having to retrain the PCA model each time.

8.3 Cluster Count Selection

Selecting the optimal number of clusters k is a multi-signal analysis. The following metrics are tracked across a range of k values:

Metric	What it Measures	Desired Behaviour
FCM Objective	Within-cluster inertia	Elbow point
Silhouette Score	Cluster separation (hard labels)	Maximised
Partition Entropy	Fuzziness of assignments	Balanced (not too crisp or diffuse)
FPC (Fuzzy Partition Coefficient)	Assignment confidence	Maximised



The final k is selected where the elbow in the FCM objective aligns with a high silhouette score and acceptable partition entropy. This multi-signal approach avoids over-optimising for a single metric.

8.4 Artefacts

The following artefacts are persisted after clustering:

- fcm_centers.npy — cluster centre vectors in PCA-reduced space.
- membership_matrix.npy — per-document soft membership matrix (n_docs × k).

- pca_model.joblib — fitted PCA model for query-time projection.

9. Semantic Cache

9.1 Motivation

A conventional cache maps an exact query string to a result. This fails to exploit the fact that two semantically equivalent queries ("gun control arguments" and "debate around firearms regulation") should return the same results. The semantic cache addresses this by keying on query embeddings rather than query strings.

9.2 Cache Lookup Algorithm

Given an incoming query:

12. Embed the query using all-MiniLM-L6-v2.
13. Project the embedding to PCA space and apply FCM to produce membership vector m.
14. Identify the dominant cluster(s) where m exceeds a membership threshold.
15. Retrieve only cache entries that share a dominant cluster — this is the cluster-gated candidate set.
16. Compute cosine similarity between the query embedding and each candidate's stored embedding.
17. If the highest similarity exceeds the threshold (default: 0.85), return the cached result as a hit.
18. Otherwise, execute full ChromaDB retrieval, store the result in cache, and return.

9.3 Cluster-Gated Search Optimisation

Rather than comparing the incoming query against every cache entry (O(n) full scan), the cluster-gated approach restricts comparisons to entries in the same dominant cluster(s). For a cache with n entries across k clusters, the expected number of comparisons per lookup is approximately n/k.

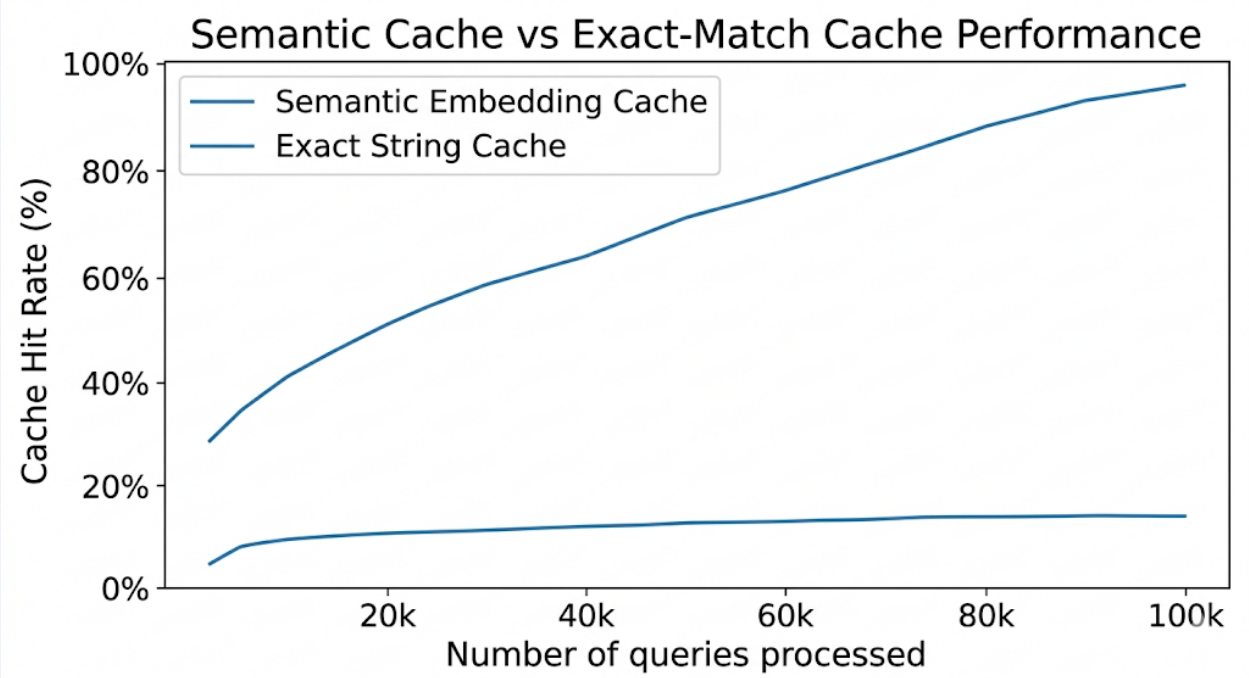
Cache Size	k (clusters)	Comparisons (O(n))	Comparisons (gated, O(n/k))
10,000	15	10,000	~667
10,000	20	10,000	~500
50,000	15	50,000	~3,333

This makes the semantic cache suitable for production workloads where the cache grows large.

9.4 Cache Policy

The cache implements the following production-grade policies:

- LRU (Least Recently Used) eviction — bounded by CACHE_MAX_SIZE (default: 10,000 entries).
- Thread safety — all read and write operations are protected by a reentrant lock (threading.RLock).
- Hit/miss counters — exposed via the /cache/stats API endpoint for monitoring.
- Cache flush — the /cache endpoint (DELETE method) clears all entries for debugging or reset.



10. API Reference (FastAPI)

The backend is served via FastAPI with uvicorn. API documentation is auto-generated at http://localhost:8000/docs.

Endpoint	Method	Description
/query	POST	Embed query, check cache, retrieve documents
/cache/stats	GET	Return hit count, miss count, cache size

/cache	DELETE	Flush the semantic cache
/health	GET	Liveness check

10.1 /query Request & Response

Request body:

```
{ "query": "What are the arguments about gun control?" }
```

Response — cache miss:

```
{ "query": "...", "cache_hit": false,
  "result": { "retrieved_documents": [...], "dominant_cluster": 3 } }
```

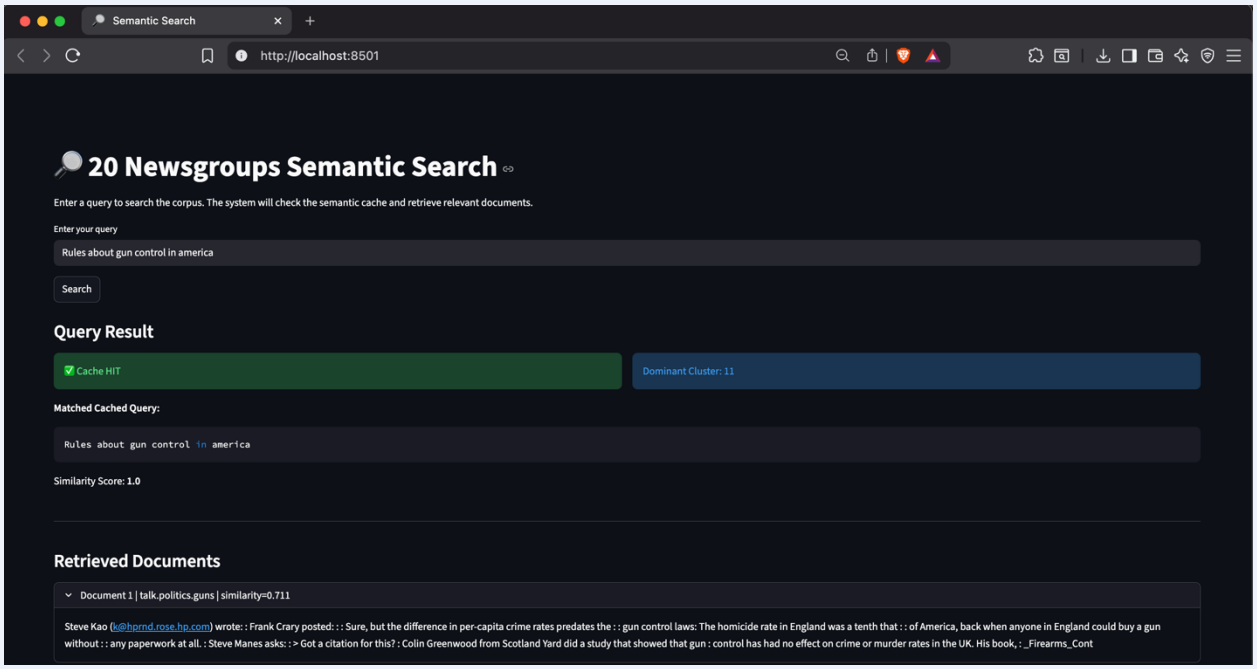
Response — cache hit:

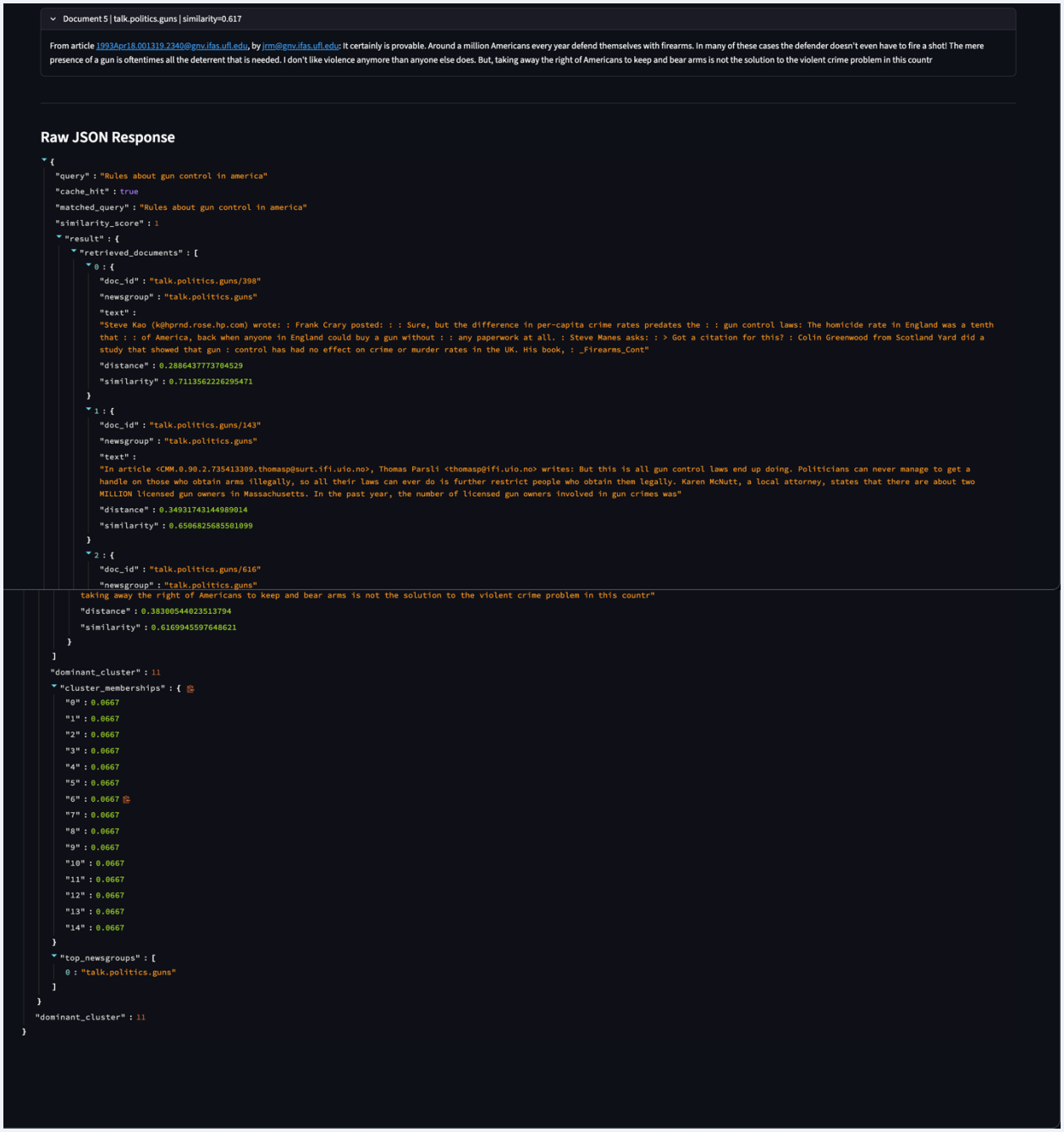
```
{ "cache_hit": true, "matched_query": "...", "similarity_score": 0.91 }
```

---

11. Frontend (Streamlit)

The Streamlit frontend provides a browser-based interface for submitting queries and viewing results. It communicates with the FastAPI backend over HTTP.





UI features:

- Query input box.
- Cache hit / miss indicator with similarity score on hits.
- Retrieved document cards showing text excerpt, newsgroup category, and relevance score.
- Dominant cluster label for the query.
- Cache statistics panel (hit count, miss count, cache size).

12. Infrastructure & Deployment

12.1 One-Command Launcher

The start.py launcher automates the full startup sequence:

19. Install dependencies from requirements.txt.
20. Check whether pipeline artefacts exist; if not, run the full offline pipeline.
21. Start the FastAPI backend (uvicorn) as a subprocess.
22. Poll /health until the backend is ready.
23. Launch the Streamlit frontend.

```
python start.py
```

12.2 Docker & Docker Compose

A Dockerfile packages the application with all Python dependencies. docker-compose.yaml orchestrates both the API and frontend services, with a shared /app/data volume mount for persistent artefacts.

Service	Port	Description
FastAPI (uvicorn)	8000	Backend API + /docs auto-documentation
Streamlit	8501	Frontend UI

12.3 Environment Variables

Variable	Default	Description
CHROMA_PERSIST_DIR	data/chroma_db	ChromaDB storage path
CLUSTERING_ARTIFACTS_DIR	data	FCM / PCA artefact path
CACHE_SIMILARITY_THRESHOLD	0.85	Cosine similarity threshold for cache hit
CACHE_MAX_SIZE	10000	Maximum number of cache entries

13. Testing

Tests are implemented using pytest and cover the following scenarios:

Test Case	Description
Cache hit	Paraphrased query returns a cached result above the similarity threshold
Cache miss	Dissimilar query correctly falls through to ChromaDB retrieval
Paraphrase detection	Semantically equivalent queries share cached results
LRU eviction	Oldest entries are evicted when cache exceeds CACHE_MAX_SIZE
Concurrency safety	Concurrent API requests do not corrupt cache state
Query engine	Embedding + retrieval returns correct top-k documents

```
pytest tests/
```

14. Performance & Runtime

14.1 Pipeline Runtime (CPU)

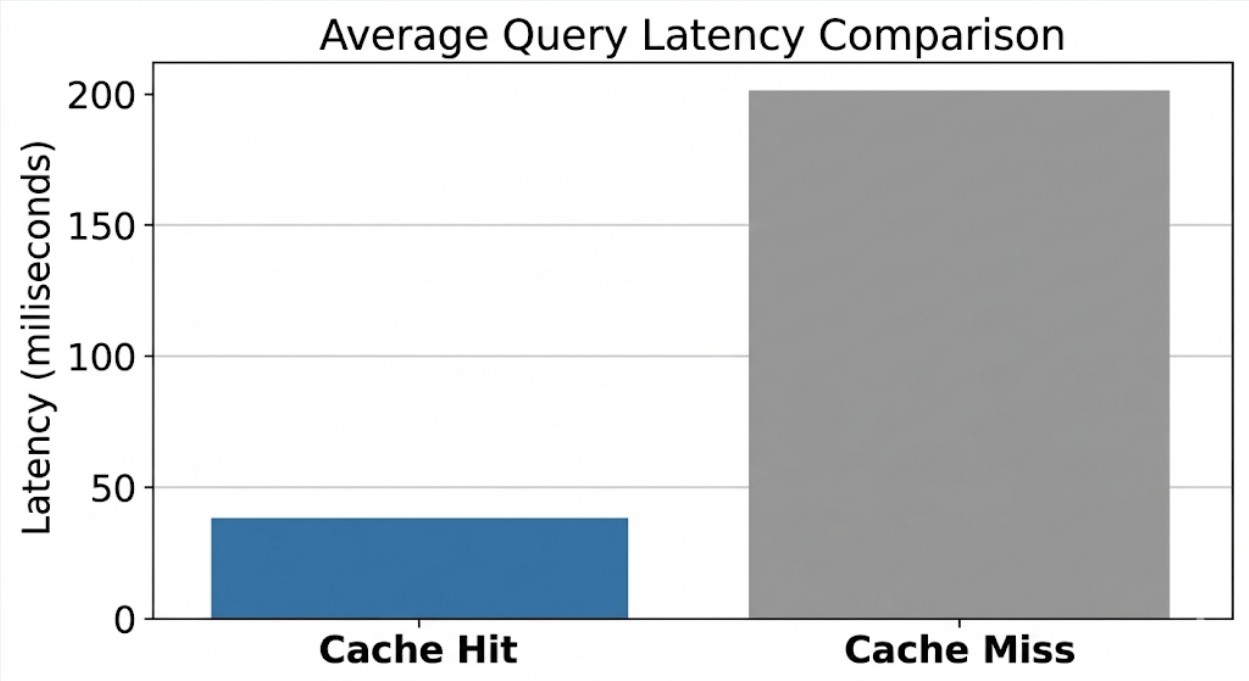
Stage	Estimated Time (CPU)
Dataset download	1–2 minutes
Text cleaning	1–2 minutes
Embedding (~18K docs)	20–40 minutes
ChromaDB ingestion	3–5 minutes
PCA + FCM clustering	5–15 minutes
Total	30–60 minutes

With a GPU, total pipeline time reduces to approximately 5–10 minutes, with the embedding stage benefiting most from GPU acceleration.

14.2 Query Latency

At query time:

- Cache hit: typically < 50ms (embedding + cluster lookup + cosine similarity).
- Cache miss: 100–300ms on CPU (embedding + ChromaDB ANN search + cache write).



15. Key Design Decisions

Decision	Choice Made	Rationale
Embedding model	all-MiniLM-L6-v2	Strong quality/speed trade-off; 384-d normalised vectors
Vector store	ChromaDB	Persistent local setup; built-in cosine similarity; metadata filters

Clustering algorithm	Fuzzy C-Means (not k-Means)	Documents span multiple topics; soft assignments are more faithful
Pre-clustering reduction	PCA 384 → 50 dims	Stabilises FCM convergence; reduces iteration cost
k selection	Multi-metric (FCM obj, Silhouette, Entropy, FPC)	Avoids over-fitting to a single metric
Cache key type	Semantic embedding (not string)	Paraphrase-tolerant; serves identical results for equivalent queries
Cache lookup	Cluster-gated (O(n/k))	Scales to large caches; avoids O(n) full scan
Cache eviction	LRU	Bounded memory; retains recently relevant entries
Concurrency	threading.RLock	Safe for multi-threaded FastAPI request handling

16. Future Improvements

- FAISS ANN search — replace ChromaDB's internal index with FAISS for sub-linear approximate nearest neighbour search at scale.
- Redis distributed cache — replace the in-process Python cache with Redis for multi-instance deployments.
- GPU embedding — batch GPU inference for 10× throughput during pipeline and reduced query latency.
- Query analytics dashboard — track query volume, cache hit rates, cluster distribution, and retrieval quality over time.
- Cluster visualisation — UMAP 2D projection of the document embedding space, coloured by FCM dominant cluster.
- Re-ranking — add a cross-encoder re-ranker pass to improve top-k result ordering.
- Streaming responses — FastAPI streaming endpoint for progressive result delivery on large result sets.

17. Project Structure

Path	Description

start.py	One-command system launcher
src/corpus_prep.py	Corpus cleaning, embedding, ChromaDB ingestion
src/fuzzy_clustering.py	PCA, FCM, k-selection, artefact saving
src/semantic_cache.py	LRU semantic cache with cluster-gated lookup
src/query_engine.py	Query embedding and retrieval
src/main.py	FastAPI application, endpoints, lifecycle
frontend/app.py	Streamlit frontend UI
scripts/download_dataset.py	Dataset downloader
scripts/run_pipeline.py	Full offline pipeline orchestration
tests/	pytest test suite
Dockerfile	Container image definition
docker-compose.yaml	Multi-service Docker Compose configuration
.env.example	Environment variable template
requirements.txt	Python dependency list

18. Conclusion

This project demonstrates the end-to-end construction of a production-grade semantic search system. The key contributions are:

24. A complete offline ML pipeline: cleaning, dense embedding, vector ingestion, and fuzzy clustering.
25. A novel semantic cache that exploits FCM cluster membership to route and gate similarity lookups, reducing average comparisons from O(n) to O(n/k).
26. A thread-safe, LRU-bounded cache implementation suitable for concurrent API workloads.
27. A deployable system with a FastAPI backend, Streamlit UI, Docker containerisation, and a one-command launcher.

The use of Fuzzy C-Means rather than hard k-means clustering is the central algorithmic contribution: soft assignments more accurately reflect the multi-topic nature of newsgroup posts and enable richer cache routing logic. The multi-metric k-selection analysis ensures the cluster structure is meaningful rather than arbitrarily chosen.

The system is designed to be extensible: FAISS ANN search, Redis-backed caching, and GPU embeddings are natural next steps that would bring the architecture closer to production readiness at scale.

— End of Report —